API Design: ExplainIt V1

Oct 19, 2026 · @Kaushtubh Mishra

Overview and conventions

The web app exposes one small JSON API, used only by its own pages; the render worker never calls it and instead works through database functions (last section). Table and column details live in the Database Design doc.

Convention	Rule
Where	Next.js App Router route handlers in `apps/web/app/api/**/route.ts`, same origin as the pages
Format	JSON in and out, `camelCase` keys, UTC ISO 8601 timestamps, UUID ids
Validation	Every request body is parsed with a Zod schema from `packages/shared/src/api.ts`; the same schemas type the browser's calls
Writes	All writes go through these routes, so the server validates scene plans and versions. The browser reads directly from Supabase only for Realtime job updates
Success codes	`200` read or update, `201` created, `202` accepted for background work, `204` deleted
Errors	One shape, `{ "error": { "code", "message", "details?" } }`, listed in Errors, rate limits, and quotas
Long calls	Script and revise calls take 30–60 s, so their handlers set `maxDuration = 120`; the page shows progress messages while waiting
Versioning	No URL version in V1; breaking changes ship with the pages that use them

Authentication and authorization

Supabase Auth signs the user in, and every route works with that user's own session, so database row-level security (RLS) enforces ownership a second time.

- **Sign-in:** Google or email magic link through Supabase Auth on `/login`. The session lives in secure HTTP-only cookies managed by `@supabase/ssr`.
- **Middleware:** `apps/web/middleware.ts` refreshes the session on each request and redirects signed-out users from `/projects/**` to `/login`.
- **Every route:** creates a server Supabase client from the request cookies. No session means `401 unauthorized`.
- **Ownership:** queries run as the user, so RLS returns only their rows. A project that belongs to someone else looks exactly like a missing one: `404 not_found`, never `403`, so ids cannot be probed.
- **Secret keys:** Gemini and R2 keys are read only inside route handlers. The Supabase service role key and ElevenLabs key never exist in the web app.
- **Individuals only:** there are no teams, roles, or sharing in V1.

Endpoint summary

Fifteen routes cover the whole V1 product, grouped by what they act on.

Method	Path	Does	Success
POST	`/api/projects`	Create a project from an idea, notes, or brainstorm mode	201
GET	`/api/projects`	List the user's projects for the library	200
GET	`/api/projects/:id`	Get one project with its scene plan	200
PATCH	`/api/projects/:id`	Save edits (title, scene plan) with a version check; used by autosave	200
DELETE	`/api/projects/:id`	Delete a project, its rows, and its R2 videos	204
GET	`/api/projects/:id/messages`	Chat history for one stage	200

Method	Path	Does	Success
POST	`/api/projects/:id/brainstorm`	One brainstorm chat turn	200
POST	`/api/projects/:id/script`	Generate the scene plan	200
POST	`/api/projects/:id/revise`	Apply a plain-language change request	200
POST	`/api/projects/:id/render`	Queue a render job for the current plan	202
GET	`/api/jobs/:id`	Job status and progress (polling fallback)	200
POST	`/api/jobs/:id/retry`	Requeue a failed job, keeping cached audio	202
POST	`/api/jobs/:id/cancel`	Cancel a job that has not started	200
GET	`/api/videos/:id/download`	A 15-minute signed R2 download link	200
GET	`/api/status`	Usage counts and whether the laptop worker is online	200

Projects

A project holds one video idea from planning to download; its scene plan is saved with a version number so two tabs cannot overwrite each other.

Create · `POST /api/projects`

```
{ "language": "hi-IN", "durationTarget": 90, "theme": "auto", "inputMode":
"notes",
  "inputText": "water vapour rises, cools, forms clouds..." }
```

`inputMode` is `idea`, `notes`, or `brainstorm`; `inputText` is up to 5,000 characters and may be empty for brainstorm. theme is auto, sky, forest, sunset, or night (default auto, which lets Gemini choose); durationTarget is 60 or 90 seconds (default 60); the script is written to fit it, and it can be changed with PATCH before scripting. Returns `201` with the project (`stage: "planning"`, `planVersion: 0`, `scenePlan: null`).

List · `GET /api/projects` returns the newest 50, each with its latest video's state for the library card:

```
{ "projects": [ {
  "id": "8f0c…", "title": "How Does Water Evaporate?", "language": "en-IN",
"stage": "ready",
  "updatedAt": "2026-10-19T09:12:04Z",
  "latestVideo": { "id": "c41a…", "status": "ready", "expiresAt": "2026-10-
19T18:29:00Z" },
  "activeJob": null
} ] }
```

`latestVideo.status` is `ready`, `expires_tonight` (under 3 hours left), or `expired`.

Get · `GET /api/projects/:id` returns the full project: `id, title, language, inputMode, inputText, scenePlan, planVersion, stage, createdAt, updatedAt, activeJob, latestVideo`.

Save edits · `PATCH /api/projects/:id`, called by autosave 1 second after the user stops editing:

```
{ "planVersion": 4, "scenePlan": { "version": 1, "title": "…", "scenes": [ …
] } }
```

The server validates the plan with the shared Zod schema and rules, then saves it only if the stored version is still 4. Responses: `200 { "planVersion": 5 }`; `409 version_conflict` with the newer plan in `details`, so the page can reload it; `422 validation_failed` with the failing fields. A `title`-only body skips plan validation.

Delete · `DELETE /api/projects/:id` refuses with `409 job_active` while a render is running; otherwise it deletes the project's R2 videos, then the project (its messages, jobs, and video rows cascade). Returns `204`.

AI endpoints

Three routes call Gemini through the shared LLM adapter; each one logs a row in `llm_calls`, which is how the hourly limit is counted.

Brainstorm · `POST /api/projects/:id/brainstorm`

```
{ "message": "something about winter" }
```

```
{ "reply": "Do you mean how snow forms, why snowflakes are unique, or why it
snows in some places?",
  "confirmedTopic": null }
```

Both turns are saved to `messages` with `stage: "brainstorm"`. When the user agrees on a topic, `confirmedTopic` holds it and the project's `inputText` is set to it, so the page can move to scripting.

Generate script · `POST /api/projects/:id/script` (no body; uses the project's input and language)

```
{ "scenePlan": { "version": 1, "title": "How Does Water Evaporate?",
"scenes": [ … ] },
  "planVersion": 1,
  "report": { "provider": "gemini:gemini-flash-latest", "attempts": 1,
"seconds": 33,
              "words": 172, "warnings": [] } }
```

Runs `generatePlan()`: Gemini Flash, then Flash-Lite, with up to 2 repair calls and automatic shortening. On success the project moves to `stage: "storyboard"`. If no valid plan comes back: `502 script_failed` with the last problems in `details`. If both models are unavailable: `503 llm_unavailable`. Calling it again replaces the plan (the page asks first).

Revise · `POST /api/projects/:id/revise`

```
{ "message": "make scene 3 simpler for kids", "stage": "storyboard",
"planVersion": 5 }
```

```
{ "scenePlan": { … }, "planVersion": 6, "changedSceneIds": ["s3"],
  "reply": "I simplified scene 3 and kept the other scenes the same." }
```

`stage` is `storyboard` or `video`. The server checks that untouched scenes really are unchanged before saving, and lists only real changes in `changedSceneIds`. A stale `planVersion` gets `409 version_conflict`. At the video stage, the page then calls render, and only the changed scenes are re-voiced.

Render, jobs, and videos

Rendering is background work: the API only queues a job and hands out download links, while the laptop worker does the rest.

Queue a render · `POST /api/projects/:id/render`

```
{ "planVersion": 6, "voice": "elevenlabs" }
```

`voice` is `elevenlabs` or `edge` (the free voice). The server checks, in order: the plan is valid and still at version 6; no other job is active for this project (`409 job_active`); the daily render limit (`429 rate_limited`); and, for ElevenLabs, that the worker's last reported credits cover the plan's characters (`422 voice_quota_low` , with the free voice offered in `details`). It then saves a job with a frozen copy of the plan and returns:

```
{ "job": { "id": "5b2e…", "status": "queued", "progress": 0 },
  "workerOnline": false,
  "warning": "Your laptop renderer is offline. The video will start when it
comes back." }
```

Job status · `GET /api/jobs/:id`

```
{ "id": "5b2e…", "status": "rendering", "progress": 45, "attempts": 1,
  "error": null, "videoId": null, "createdAt": "…", "finishedAt": null }
```

`status` is `queued` , `voicing` , `rendering` , `uploading` , `done` , `failed` , or `cancelled` . Pages normally get these changes live through Supabase Realtime on the user's `jobs` rows, and poll this route every 3 seconds only if the live connection drops.

Retry · `POST /api/jobs/:id/retry` works only on `failed` jobs and puts them back to `queued` ; cached audio means voiced scenes are not voiced again. **Cancel** · `POST /api/jobs/:id/cancel` works only while `queued` ; a running job finishes, because stopping a render midway saves nothing.

Download · `GET /api/videos/:id/download`

```
{ "url": "https://…r2.cloudflarestorage.com/…?X-Amz-Signature=…",
"expiresInSeconds": 900,
  "fileName": "how-does-water-evaporate.mp4", "videoExpiresAt": "2026-10-
19T18:29:00Z" }
```

The link is signed for 15 minutes and sets the download file name. A deleted video returns `410 video_expired`, and the page offers "Render again".

Status · `GET /api/status` gives the header and render button what they need:

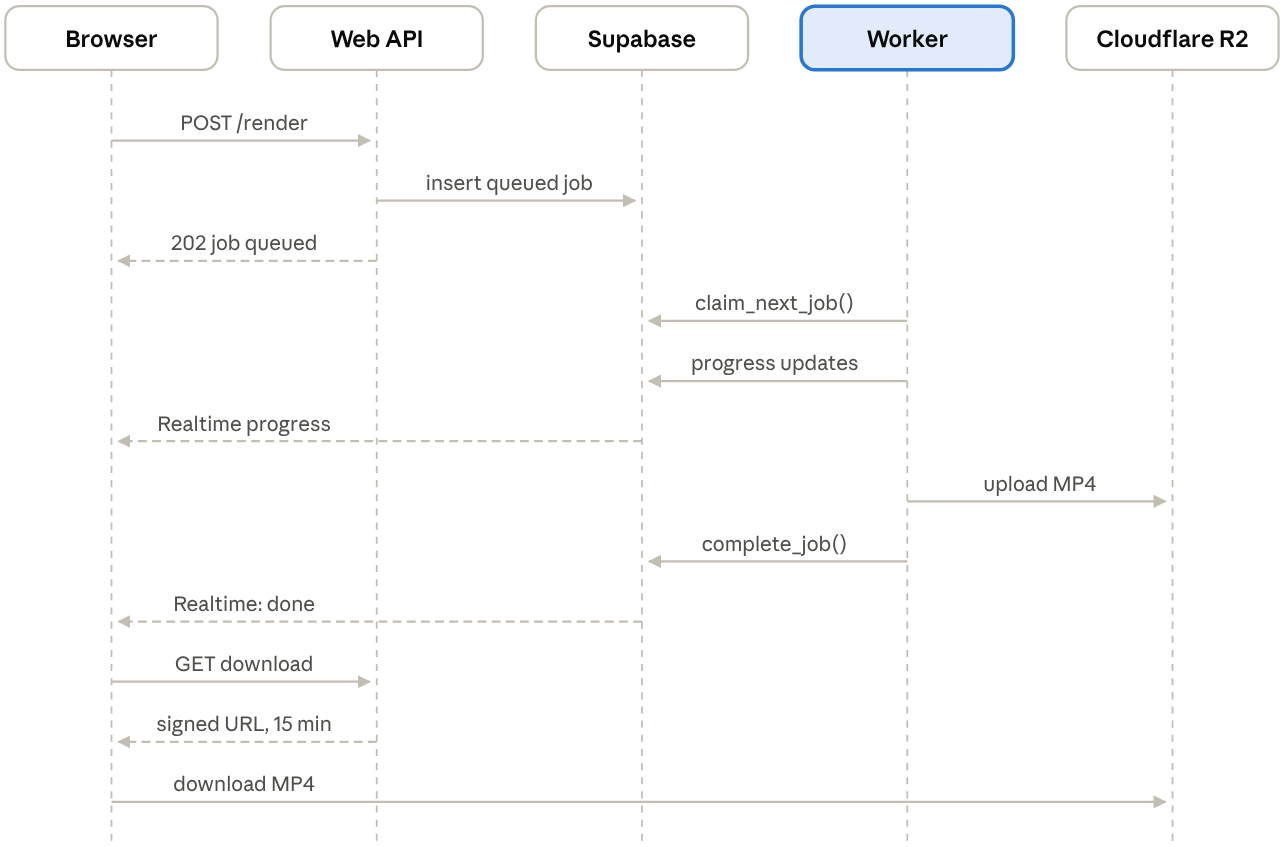
```
{ "worker": { "online": true, "lastSeenAt": "…", "voiceCreditsLeft": 7420 },
  "usage": { "llmCallsThisHour": 3, "llmCallsPerHour": 20, "rendersToday": 1,
"rendersPerDay": 5 } }
```

The worker counts as online if it checked in within the last 60 seconds.

Render flow

One render, from the click to the download, touches the API only three times; everything in between happens through the database.

The API queues a render; the worker does it



Render flow · 5 participants, 12 messages

Dashed arrows are replies or pushed updates. Because the job sits in the database, the click succeeds even when the laptop worker is off; the video starts when the worker comes back.

Errors, rate limits, and quotas

Every error uses the same shape, and every code has one fixed meaning the pages can act on.

```
{ "error": { "code": "version_conflict", "message": "This project changed in another tab.",
             "details": { "planVersion": 7 } } }
```

Code	HTTP	Meaning	What the page does
`unauthorized`	401	No valid session	Go to `/login`
`not_found`	404	No such row for this user	Show "Not found" and a link to the library
`validation_failed`	422	Body or scene plan fails its schema or rules	Highlight `details.fields`
`version_conflict`	409	The plan changed since it was loaded	Reload the newer plan and tell the user
`job_active`	409	A render is already running for the project	Show that job's progress
`voice_quota_low`	422	ElevenLabs credits do not cover the plan	Offer the free voice
`rate_limited`	429	A usage limit was reached; `Retry-After` header set	Show when it resets
`video_expired`	410	The MP4 was deleted at day end	Offer "Render again"
`script_failed`	502	Gemini answered but no valid plan after repairs	Ask to rephrase the topic
`llm_unavailable`	503	Flash and Flash-Lite both unavailable	Ask to try again in a minute

Code	HTTP	Meaning	What the page does
`internal`	500	Anything unexpected; logged with a request id	Show a generic message with the id

Limits per user, counted from database rows so they survive restarts:

Limit	Value	Counted from
LLM calls (brainstorm, script, revise)	20 per rolling hour	`llm_calls`
Render jobs	5 per day (IST)	`jobs` created today, excluding `cancelled`
Pasted notes	5,000 characters	Request validation
Change requests	1,000 characters	Request validation

These limits protect the free tiers: Gemini's per-project quota and about 10,000 ElevenLabs credits a month.

Worker contract

The worker talks only to Supabase, with the service role key, through four database functions and plain row updates; the web app never calls the worker, and the worker never calls the web app.

Call	When	Effect
`rpc('claim_next_job')`	Every 5 s while idle	Atomically moves the oldest `queued` job to `voicing` and returns it with its frozen plan, or nothing
`update jobs set status, progress`	After each scene voiced and every 5% of rendering	Live progress; Realtime pushes the change to the browser

Call	When	Effect
`rpc('complete_job', job_id, r2_key, size_bytes, duration_seconds, metrics)`	After the upload succeeds	Marks the job `done`, inserts the `videos` row with tonight's expiry, sets the project to `ready`
`rpc('fail_job', job_id, error, metrics)`	On any stage error after its retries	Marks the job `failed` with a plain-language error
`rpc('requeue_stale_jobs', 30)`	On worker start	Puts jobs stuck in a working status for over 30 minutes back to `queued`
`rpc('worker_heartbeat', voice_credits_left, version)`	Every 30 s	Lets `/api/status` show online or offline and the voice credits left
Select expired `videos`, delete from R2, set `deleted_at`	23:59 IST and on start	Day-end cleanup

The frozen plan matters: a job renders exactly the plan approved when Render was clicked, even if the user keeps editing. Function definitions are in the Database Design doc.